《人工智能与Python》——人工智能导论

工智能与Python程序设计 教研组

关于逻辑斯蒂回归梯度的推导

逻辑回归损失函数的构造

- 什么样的参数 $\boldsymbol{w}$会使得观察到训练数据$D=\{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \dots, (\boldsymbol{x}_N, y_N)\}$的概率最大?
  - 对于单个训练数据$(\boldsymbol{x}_i, y_i)$
    - 如果为正例： $y_i = 1$， 使得p_i最大
    - 如果为负例： $y_i = 0$， 使得$1 - p_i$最大

⇩

    - 总体而言：使得$p_i^{y_i} \times (1 - p_i)^{1-y_i}$最大
      - 取对数：使得$y_i \ln p_i + (1 - y_i) \ln(1 - p_i)$ 最大
  - 对于N个训练数据$D=\{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \dots, (\boldsymbol{x}_N, y_N)\}$， 使得它们的联合概率越大（负联合概率最小）

$$L(\boldsymbol{w}) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$$

逻辑回归

- 损失函数

$$L(\boldsymbol{w}) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \ln p_i + (1-y_i)\ln(1-p_i)]$$

  - 一般称为交叉熵损失函数
  - 应用梯度下降法：梯度（j 是维度索引）

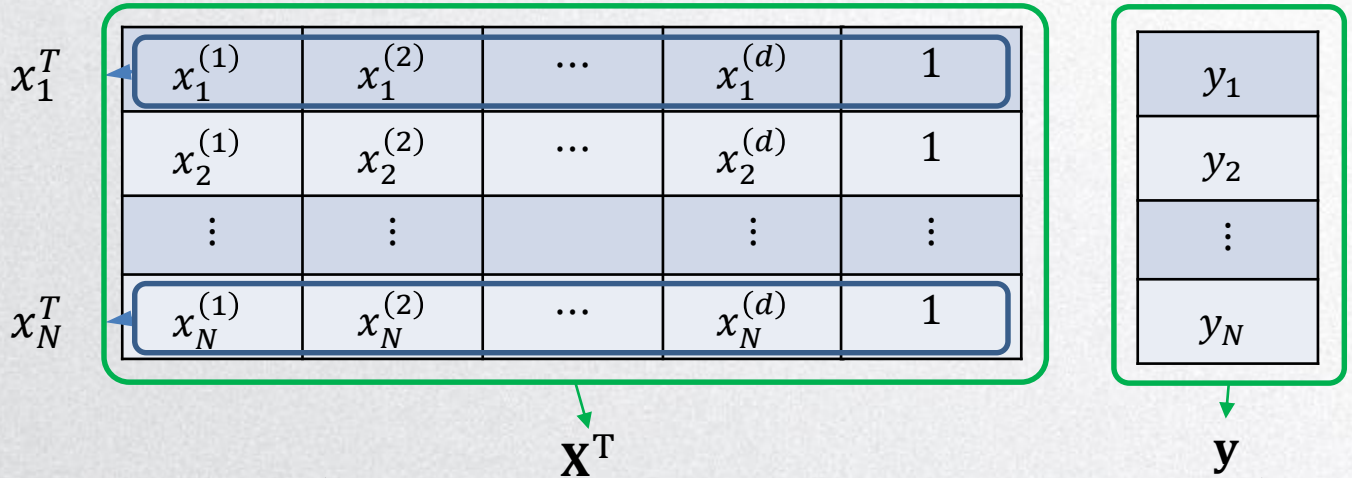
$$\frac{\partial L(\boldsymbol{w})}{\partial w_j} = \frac{1}{N}\sum_{i=1}^{N} x_i^{(j)}(p_i - y_i)$$

其中$p_i = \frac{1}{1+e^{-z_i}} = \frac{1}{1+e^{-(\boldsymbol{w^T x_i})}}$

逻辑回归梯度的具体推导

- Sigmoid函数导数：$p = g(z) = \frac{1}{e^{-z}+1} \Rightarrow \frac{dp}{dz} = \frac{dg}{dz} = g(z)\big(1-g(z)\big) = p(1-p)$
  - $\frac{dp_i}{dw^j} = \frac{dg(\mathbf{w^T x_i})}{dw^j} = g\big(\mathbf{w^T x_i}\big)\Big(1-g\big(\mathbf{w^T x_i}\big)\Big)\frac{d(\mathbf{w^T x_i})}{dw^j} = p_i(1-p_i)x_i^j$
- $\frac{\partial L(\boldsymbol{w})}{\partial w_j} = -\frac{\partial \frac{1}{N}\sum_{i=1}^{N}[y_i \ln p_i + (1-y_i)\ln(1-p_i)]}{\partial w_j}$
- $= -\frac{1}{N}\sum_{i=1}^{N}\left[\frac{y_i}{p_i}\frac{dp_i}{dw^j} - \frac{1-y_i}{1-p_i}\frac{dp_i}{dw^j}\right]$
- $= -\frac{1}{N}\sum_{i=1}^{N}\left[\frac{y_i}{p_i} - \frac{1-y_i}{1-p_i}\right]\frac{dp_i}{dw^j}$
- $= -\frac{1}{N}\sum_{i=1}^{N}\left[\frac{y_i}{p_i} - \frac{1-y_i}{1-p_i}\right]p_i(1-p_i)x_i^j$
- $= -\frac{1}{N}\sum_{i=1}^{N}[y_i(1-p_i) - (1-y_i)p_i]x_i^j$
- $= \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)x_i^j$

梯度的矩阵形式



$$\frac{\partial L(\boldsymbol{w})}{\partial \boldsymbol{w}} = \left(\frac{1}{N}\sum_{i=1}^{N} x_i^{(1)}(p_i - y_i), \frac{1}{N}\sum_{i=1}^{N} x_i^{(2)}(p_i - y_i), \cdots, \frac{1}{N}\sum_{i=1}^{N} x_i^{(d+1)}(p_i - y_i) \right)$$

- 矩阵形式：

$$\frac{\partial L(\boldsymbol{w})}{\partial \boldsymbol{w}} = \frac{1}{N}\mathbf{X}^{\mathrm{T}}(\boldsymbol{p} - \mathbf{y})$$

注意：此处$\mathbf{X}$的维度是$N \times (d+1)$，$\boldsymbol{p} - \boldsymbol{y} = \begin{bmatrix} p_1 - y_1 \\ \vdots \\ p_N - y_N \end{bmatrix}$维度为$N \times 1$

半期考试题目讲解

题目：实现函数find_letter

实现find_letter函数，其输入为全为小写字母或者数字的字符串。请找出第一个只出现一次的字符，并返回索引，如果这样的字符不存在返回None。

```
def find_letter(a):
    # 补全该函数
```

例如：

调用： find_letter("aabbcc2233dd") 返回： None

调用： find_letter("112abcd") 返回： (2, "2")

调用： find_letter("babccde") 返回： (1, "a")

调用： find_letter("abcdefg") 返回： (0, "a")

调用： find_letter("998") 返回： (2, "8")

调用： find_letter("") 返回： None

题目：实现函数num_dates

实现num_dates函数，其输入一个整数型列表，每一个数字表示每一天的日气温，按照日期从小到大排列；输出为从对应的日期算起，多少天后气温才会升高；如果之后都不会升高，对应位置天数为0；如果输入为空列表或者列表中有数值不为整数，则返回None

```
def num_dates(a):
    # 补全该函数
```

例如：

调用： num_dates([]) 返回： None

调用： num_dates("11", 20) 返回： None

调用： num_dates([11, 22, 33]) 返回： [1, 1, 0]

调用： num_dates([1, 2, 4, 3, 2, 5, 3]) 返回： [1, 1, 3, 2, 1, 0, 0]

调用： num_dates([1, 1, 1, 1, 1]) 返回： [0, 0, 0, 0, 0]

调用： num_dates([1, 2, 3, 2, 1, 2, 3, 4]) 返回： [1, 1, 5, 3, 1, 1, 1, 0]

题目：简化版ZigZag遍历

ZigZag遍历指的是对一个矩阵进行之字形遍历，该遍历在图形图像、视频等领域有重要应用。本题考虑一个简化版的ZigZag遍历，即将矩阵切成从右上至左下的切片，并按照示例中的顺序逐个元素输出。编写代码，对输入的矩阵进行遍历，按顺序输出列表；若输入不是矩阵形式（即内层列表长度不一致），输出空列表。

```
def simplified_zigzag(l):
    # 补全该函数
```

例如：

对如下的输入进行遍历：

```
[[1,  2,  3,  4],
 [5,  6,  7,  8],
 [9, 10, 11, 12]]
```

应输出：

```
[1, 2, 5, 3, 6, 9, 4, 7, 10, 8, 11, 12]
```

对如下的输入进行遍历：

```
[['A', 'B', 'C'],
 ['D', 'E', 'F'],
 ['G', 'H', 'I'],
 ['J', 'K', 'L'],
 ['M', 'N', 'O']]
```

应输出：

```
['A', 'B', 'D', 'C', 'E', 'G', 'F', 'H', 'J', 'I', 'K', 'M', 'L', 'N', 'O']
```

对如下的矩阵进行遍历：

```
[['A', 'B', 'C'],
 ['D', 'E', 'F'],
 ['G', 'H'],
 ['J', 'K', 'L'],
 ['M', 'N', 'O']]
```

应输出：

```
[]
```

题目：逐项相乘

逐项相乘是一个二元运算，其输入（函数参数）为两个相同形状的列表，输出（函数返回值）是具有同样形状的、各个位置的元素等于两个输入列表相同位置元素的乘积的列表。注意，列表可以嵌套，所以输入可以是一维的列表，也可以是二维嵌套列表（矩阵），甚至更高维嵌套的列表。

测试样例保证x，y的形状相同，且嵌套列表中保存的数据均为整数。

```
def elementwise_product(x, y):
    # 补全该函数
```

例如：

```
x, y = [1], [2]
print(elementwise_product(x, y))
```

应输出：

```
[2]
```

```
x, y = [1,2,3], [1,2,3]
print(elementwise_product(x, y))
```

应输出：

```
[1, 4, 9]
```

```
x, y = [[1,2,3], [2,3,1], [3, 1, 2]], [[1,0,0], [0,1,0], [0,0,1]]
print(elementwise_product(x, y))
```

应输出：

```
[[1, 0, 0], [0, 3, 0], [0, 0, 2]]
```

```
x, y = [[[1], [2]], [[3], [4]]], [[[1], [2]], [[3], [4]]]
print(product(x, y))
```

应输出：

```
[[[1], [4]], [[9], [16]]]
```

题目：整数数字逆序

实现int_reverse函数。输入一个整数，将整数中的数字逆序，返回逆序后的整数；若输入不是整数，返回None。

```
def int_reverse(data):
    # 补全该函数
```

例如：

调用： int_reverse(123)

返回： 321

调用： int_reverse(+10)

返回： 1

调用： int_reverse(-10707)

返回： -70701

调用： int_reverse('abcdg')

返回： None

题目：爬楼梯

小青蛙爬楼梯，可以每次爬 1 级或者 2 级台阶，输入楼梯的级数（正整数），返回所有可能的爬法数。例如，台阶级数为2，可能的爬法数为2（连续两次各爬一级；一次爬两级）。

```
def frog_climb_stairs(N):
    # 补全该函数
```

例如：

调用： frog_climb_stairs(1)

返回： 1

调用： frog_climb_stairs(2)

返回： 2

调用： frog_climb_stairs(3)

返回： 3

调用： frog_climb_stairs(4)

返回： 5

题目：总上升段数

一位登山爱好者从起点开始，每隔一段时间记录其海拔高度直至终点。登山结束后，他想知道在登山过程中他的总上升段数，海拔的连续上升计为一段。编写一个函数，帮这位登山爱好者计算总上升高度及上升段数，输入是一个整数列表，数据是按照时间排列的海拔高度记录，输出总上升段数。你可以假设输入的数组中至少有两个数。

```
def total_ascend(h):
    # 补全该函数
```

例如：

对如下的输入调用函数：

```
[100, 200, 150, 250]
```

应输出：

```
2
```

说明：共有两段上升：100 -> 200, 150 -> 250

对如下的输入调用函数：

```
[100, 80, 150, 250]
```

应输出：

```
1
```

说明：共有一段上升：80 -> 250

题目：实现函数count_characters

实现count_characters函数，其输入为全为英文字母（不区分大小写）的字符串。请按照字母顺序，输出每个出现的字母（以小写形式）以及对应的出现频率，不同字母的信息以“逗号+空格”隔开（行尾不需要打印空格）。

```
def count_characters(string):
    # 补全该函数
```

例如：

调用： count_characters("AAAbb") 返回： a:3, b:2

调用： count_characters("zzAAAbb") 返回： a:3, b:2, z:2

题目：实现函数reverse_even_characters

实现reverse_even_characters函数，其输入为全为数字与英文字母（不区分大小写）组成的字符串。现在需要对其进行变换，所有奇数位的字符原地不动，所有偶数位的字符进行从前到后的整体的反转。

```
def reverse_even_characters(string):
    # 补全该函数
```

例如：

调用： reverse_even_characters("a1b2c3") 返回： a3b2c1 提示：偶数位的字符为1、2、3，需要将其出现的顺序进行反转，因而得到3、2、1，并且按照此顺序输出到原来的偶数位置。

调用： count_characters("1a2b3c") 返回： 1c2b3a

题目：采用面向对象方法实现车辆管理：

有一个父类 `Auto`，具有属性 `brand` 和 `color`，即汽车的品牌与颜色

有两个子类：`Car` 和 `Bus`。它们继承自 `Auto` 类。其中，`Car` 类表示轿车，`Bus` 类表示大巴车。`Bus` 类具备一个额外的私有属性 `carry_passengers`，即载客人数。

现在，有一个 `AutoMarket` 类，管理所有汽车的实例。该类具有以下方法：

- `buy_auto(auto:Auto)`：将一个汽车实例添加到汽车市场中。
- `current_auto_storage()`：显示所有添加到汽车市场中的汽车品牌、颜色、载客人数等属性。

要求：

请你填写如下 Python 代码，采用面向对象方法，使其能够顺利运行，满足上述要求。

```
class Auto(object):
    def __init__(self, brand, color):
        self.__brand = brand
        self.__color = color

class Car(Auto): # 补全该类


class Bus(Auto):
    def __init__(self, brand, color, carry_passengers): # 补全该函数，需采用父类构造函数


class AutoMarket(object):
    def __init__(self):
        self.auto = {'Car':[], 'Bus':[]}

    def buy_auto(self, auto):  # 补全该函数

    def current_auto_storage(self):   # 补全该函数
        for car in self.auto['Car']:
            print("Having car {0}, color is {1}".format(car._Auto__brand, car._Auto__color))
```

```
def testing_AutoMarket(auto_list):
    # 用于测试，此函数禁止操作，否则记为0分！
    auto_market = AutoMarket()
    for auto in auto_list:
        auto_type = auto[0].split("_")
        if auto_type[0] == "car":
            car = Car(brand = auto[1]['brand'],
                      color = auto[1]['color'])
            auto_market.buy_auto(car)
        elif auto_type[0] == "bus":
            bus = Bus(brand = auto[1]['brand'],
                      color = auto[1]['color'],
                      carry_passengers = auto[1]['carry_passengers']
                     )
            auto_market.buy_auto(bus)
        else:
            print("Incorrect auto type!")
    auto_market.current_auto_storage()
```

使用样例：

```
auto_list = [['car_1',{'brand':'audi_A4', 'color':'red'}],
             ['car_2',{'brand':'xiaomi_su7', 'color':'blue'}],
             ['bus_1',{'brand':'Jinlong', 'color':'write', 'carry_passengers':40}],
             ['car_3',{'brand':'byd_qin','color':'black'}]]

testing_AutoMarket(auto_list)
```

输出结果：

```
Having car audi_A4, color is red
Having car xiaomi_su7, color is blue
Having car byd_qin, color is black
Having bus Jinlong, color is write, carry_passengers is 40
```



Python 0X
人工智能导论

提纲

- 人工智能的发展历史
- 人工智能的重要应用
- 人工智能的重要进展
- 展望与总结



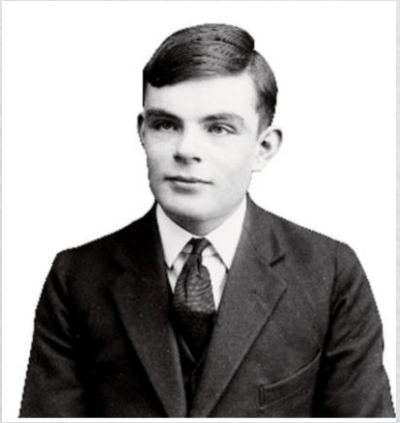
Python 0X
人工智能导论

提纲

- **人工智能的发展历史**
- 人工智能的重要应用
- 人工智能的重要进展
- 展望与总结

人工智能的发展历史

- 人工智能的诞生（20世纪40～50年代）
  - 1950年，图灵测试
  - 1954年，第一台可编程机器人诞生
  - 1956年，第一次人工智能研讨会首次提出“人工智能”概念

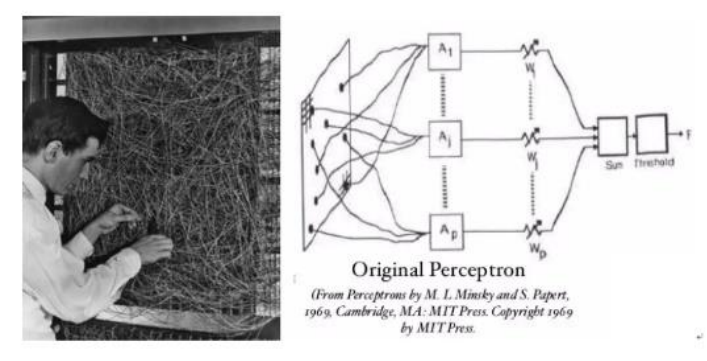






人工智能的发展历史

- 人工智能的诞生（20世纪40～50年代）
  - 1957：Frank Rosenblatt提出感知机模型，人工神经网络的雏形开始出现
  - 1959：Arthur Samuel（亚瑟·塞缪尔）完成了最高的机器学习实验，让程序学会了纯粹通过自己与自己玩来学习跳棋游戏，这是一项惊人的壮举



Original Perceptron

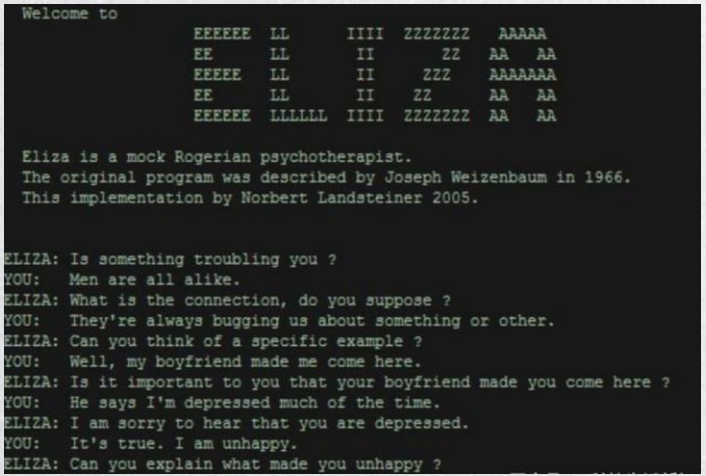
(From Perceptrons by M. L. Minsky and S. Papert, 1969, Cambridge, MA: MIT Press. Copyright 1969 by MIT Press.



人工智能的发展历史

- 人工智能的第一次黄金时代（20世纪50～70年代）
  - 1966年~1972年：首台人工智能机器人Shakey诞生
  - 1966年：世界上第一个聊天机器人ELIZA发布
  - 1968年：计算机鼠标发明

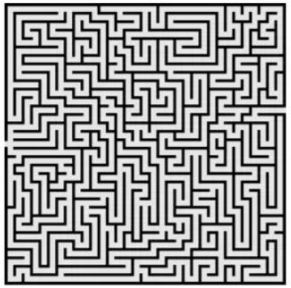






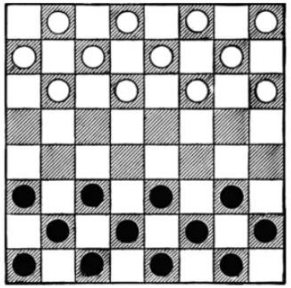
人工智能的发展历史

- 人工智能的第一次低谷（20世纪70～80年代）
  - 计算机有限的内存和处理速度不足以解决实际的人工智能问题
  - 要求程序对这个世界具有儿童水平的认识很难实现
  - 对于人工智能研究的资助逐渐减少



每多一个路口

可能路线计算量翻倍



西洋跳棋每步

可能棋局10^{20}



国际象棋每步

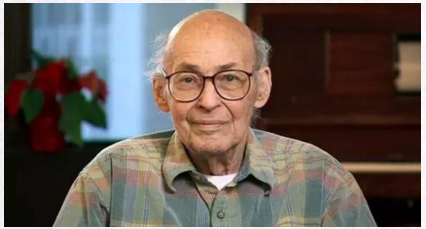
可能棋局10^{40}

人工智能的发展历史

- 人工智能的第一次低谷（20世纪70～80年代）
  - 1966：美国科学院语言自动处理咨询委员会(ALPAC)经过两年的研究，公布了一份报告否定了机器翻译的可行性
  - 1969：马文·明斯基撰写了名为《感知器》一书，指出了当时新兴神经网络领域的关键问题和缺陷
  - 1973: 小丘报告（Lighthill report）怀疑人工智能是否有必要做为一门独立学科存在



John R. Pierce (ALPAC主席)



马文·明斯基（Marvin Minsky）

上述报告把人工智能打入“冷宫”并经历了几乎十年“严冬”

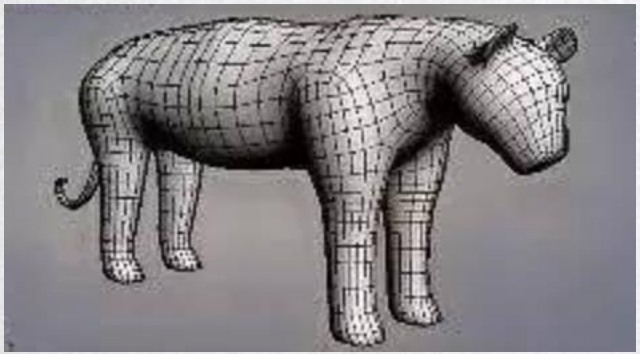
人工智能的发展历史

- 人工智能的繁荣期（1980年~1987年）
  - 以专家系统为代表
    - 采用（基于规则的）知识表示和知识推理技术来模拟通常由领域**专家**才能解决的复杂问题
    - 第一个被广泛使用的AI技术，被很多“Fortune 500“公司采用
  - 标志性的LISP机器商业化
  - 1986：反向传播算法（Backpropogation algorithm）出现

人工智能的发展历史

- 人工智能的繁荣期（1980年~1987年）
  - 1984年：启动Cyc（大百科全书）项目
  - 1986年：3D打印机问世





人工智能的发展历史

- 人工智能的第二次冬天（1987年~1993年）
  - 专家系统难以升级扩展，鲁棒性不够，导致高昂的维护成本
  - Lisp机器和Thinking Machines都做不出来
  - 神经网络效果未能超过统计的方法，资源消耗大，无法解决大规模问题
  - 日本第五代计算机计划失败
  - 政府投入缩减
  - PC普及，大型计算机受到冲击

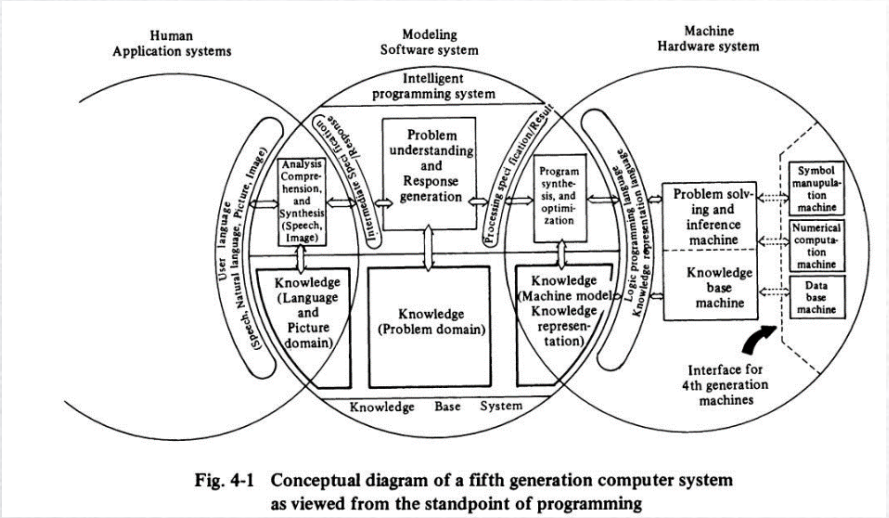


Fig. 4-1 Conceptual diagram of a fifth generation computer system as viewed from the standpoint of programming

人工智能的发展历史

- 人工智能的发展历史
  - 人工智能的第二次冬天（1987年~1993年）
    - “AI之冬”一词由经历过1974年经费削减的研究者们创造出来。
    - 专家系统的实用性仅仅局限于某些特定情景
    - 美国国防部高级研究计划局（DARPA）认为人工智能并非“下一个浪潮”



人工智能的发展历史

- 人工智能真正的春天（1993年至今）
  - 统计机器学习时代
    - 支持向量机（SVM）开启了机器学习时代
    - 被用于语音、图像、文本、机器人等各个领域



- 1997年，IBM 制造的专门超级计算机深蓝（Deep Blue）击败了国际象棋世界冠军 Garry Kasparov

- 基于AI技术的搜索引擎获得巨大的成功（Google、Yahoo!等）





人工智能的发展历史

- 人工智能真正的春天（1993年至今）
  - 深度学习崛起
    - 2006：Geoffrey Hinton发表Science论文

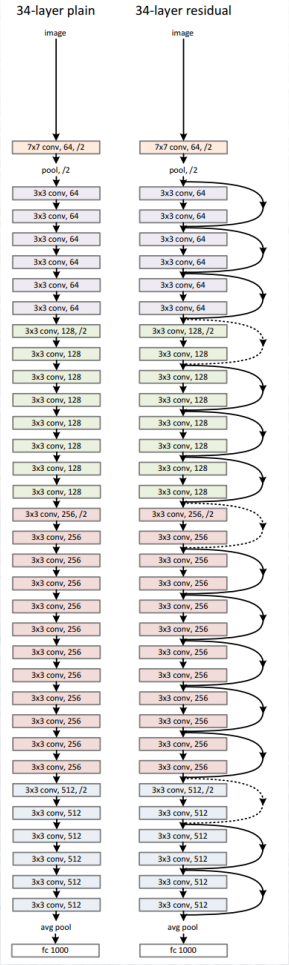


G. E. Hinton, S. Osindero, and Y. Teh, "**A fast learning algorithm for deep belief nets**," Neural Computation, vol. 18, pp. 1527–1554, 2006.

G. E. Hinton and R. R. Salakhutdniov, "**Reducing the dimensionality of data with neural networks**," Science, vol. 313, no. 5786, pp. 504-507, July 2006.

- 神经网络借深度学习之名再度登场
- 2009：ImageNet数据集发布，2010年举行首次ImageNet竞赛

IMAGENET



深度神经网络

RENMIN UNIVERSITY OF CHINA 1937 中国人民大学

人工智能的发展历史

- 人工智能真正的春天（1993年至今）
  - 重要事件汇总：
    - 1997年：电脑深蓝战胜国际象棋世界冠军
    - 2011年：开发出使用自然语言回答问题的人工智能程序
    - 2012年：Spaun诞生
    - 2013年：深度学习算法开始运用在产品开发中
    - 2015年：开源库、人工智能研究所
    - 2016年：AlphaGo战胜围棋世界冠军李世石



Python 0X
人工智能导论

提纲

- 人工智能的发展历史
- **人工智能的重要应用**
- 人工智能的重要技术
- 展望与总结

人工智能的重要应用

- 个人语音助手
  - 能够进行基本的语音交互
  - 提供较为丰富的应用服务
  - 执行固定项目的行为功能
  - 对已有应用进行融合、包装









人工智能的重要应用

- 人脸识别
  - 识别准确率较高
  - 应用非常广泛
  - 达到了大规模部署、应用的水平







人工智能的重要应用

- 智能机器人
  - 面向工业领域或者其他特定的具体应用
  - 在特定任务可以做到非常精准
  - 具有重要的工业、生活需求





人工智能的重要应用

- 自动驾驶
  - 对于智能体的综合智能能力要求较高
  - 非实战环境达到了较好效果
  - 目前仍具有很大的实践挑战





人工智能的重要应用

- 聊天机器人
  - 对于一般性问题已经达到了很好的效果
  - 长期、多轮聊天效果仍然需要提升
  - 对于常识性和背景知识仍然缺乏推理能力





人工智能的重要应用

- 游戏AI
  - 相对于通用应用，算是相对容易的任务
  - 特定任务可能达到超人类水平
  - 需要足够的计算资源支撑







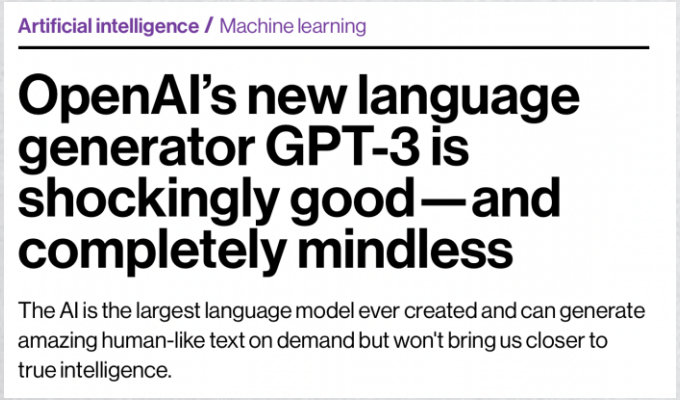
Python 0X
人工智能导论

提纲

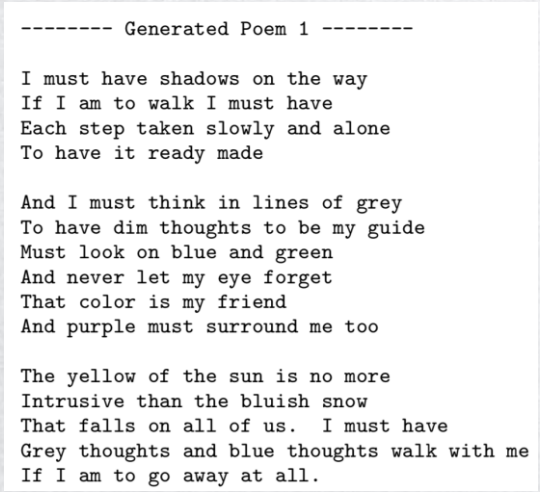
- 人工智能的发展历史
- 人工智能的重要应用
- **人工智能的重要进展**
- 展望与总结

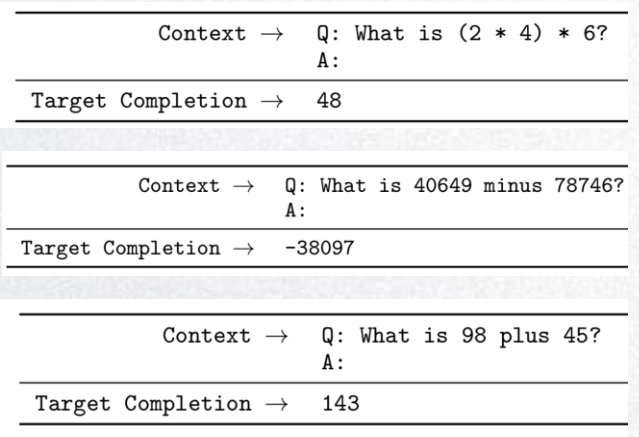
人工智能的重要进展

- 语言
  - 代表性进展（1）
    - 文本生成
      - 从传统语言模型过渡到大规模预训练神经网络语言模型



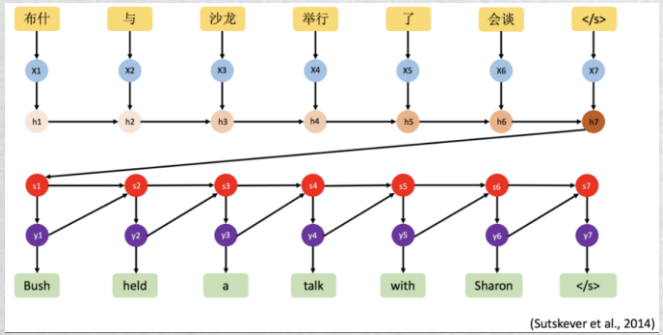
MIT technical review

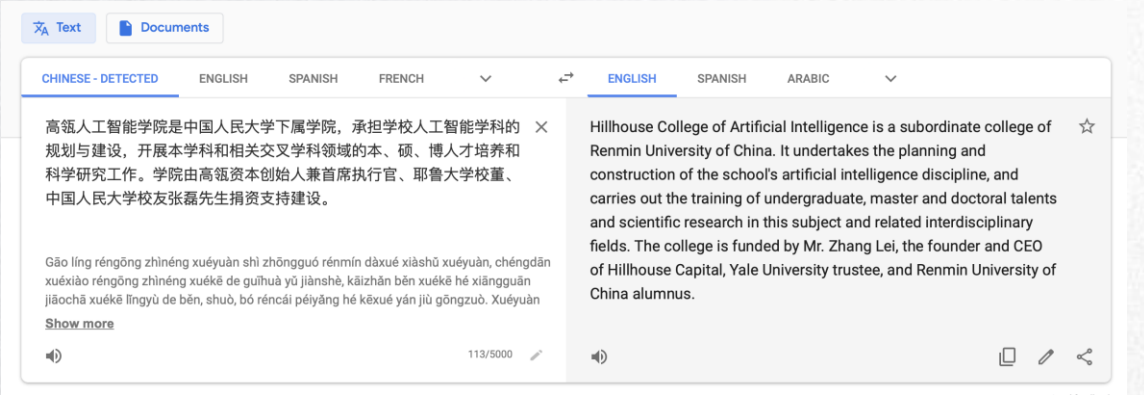




人工智能的重要进展

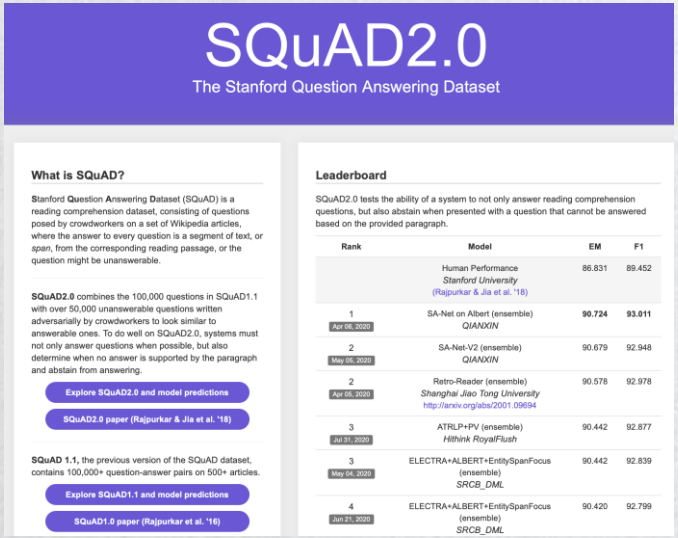
- 语言
  - 代表性进展（2）
    - 翻译系统
      - 从传统基于统计、语法规则的系统过渡到端到端的神经网络翻译模型





人工智能的重要进展

- 语言
  - 代表性进展（3）
    - 阅读理解
      - 从基于语法规则的传统系统过渡到可以大规模训练的神经网络系统



In meteorology, precipitation is any product of the condensation of atmospheric water vapor that falls under **gravity**. The main forms of precipitation include drizzle, rain, sleet, snow, **graupel** and hail... Precipitation forms as smaller droplets coalesce via collision with other rain drops or ice crystals **within a cloud**. Short, intense periods of rain in scattered locations are called "showers".

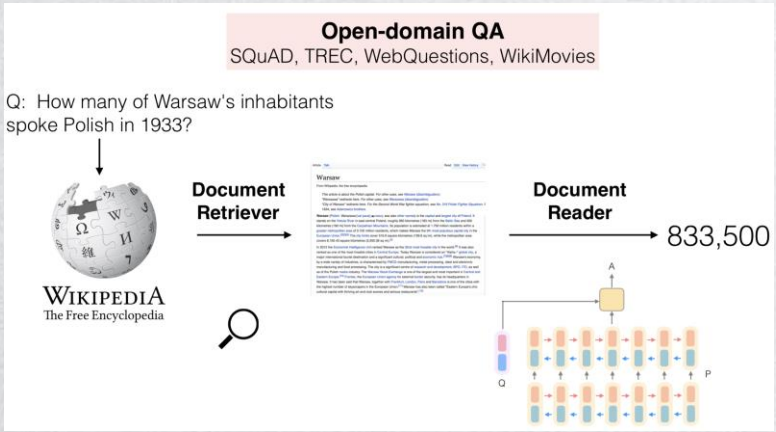
What causes precipitation to fall?
gravity

What is another main form of precipitation besides drizzle, rain, snow, sleet and hail?
graupel

Where do water droplets collide with ice crystals to form precipitation?
within a cloud

人工智能的重要进展

- 语言
  - 代表性进展（4）
    - 问答系统
      - 借助阅读理解技术以及大规模知识图谱，实现了开放域的神经网络事实型问答系统




```
>>> process('Who was the winning pitcher in the 1956 World Series?')

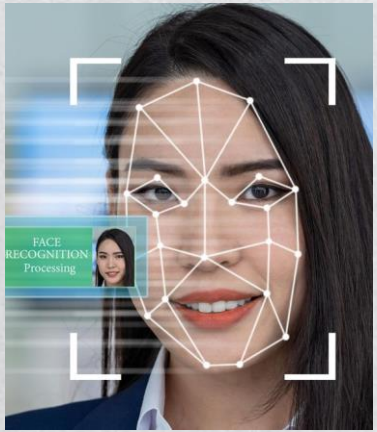
Top Predictions:
+------+-------------+-----------------+--------------+-----------+
| Rank |   Answer    |       Doc       | Answer Score | Doc Score |
+------+-------------+-----------------+--------------+-----------+
|  1   | Don Larsen | New York Yankees |  4.5059e+06  |   278.06  |
+------+-------------+-----------------+--------------+-----------+

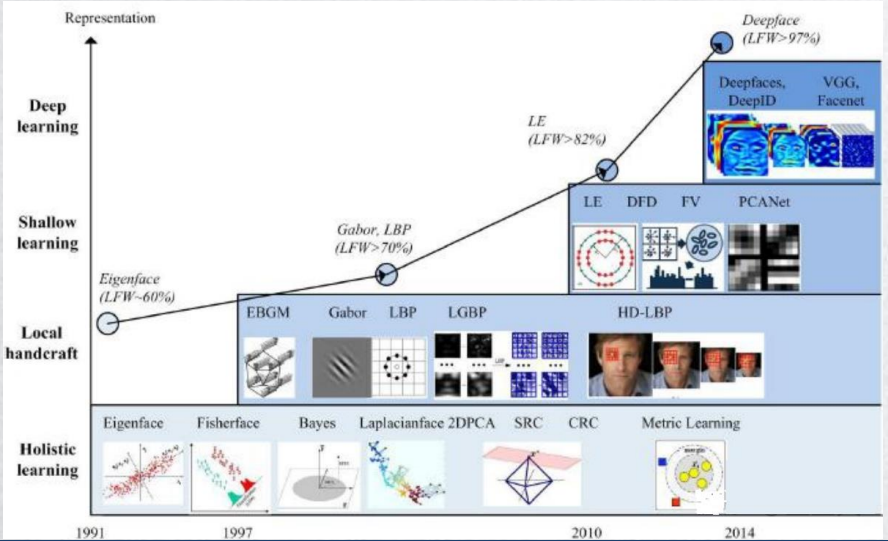
Contexts:
[ Doc = New York Yankees ]
In 1954, the Yankees won over 100 games, but the Indians took the pennant with
an AL record 111 wins; 1954 was famously referred to as "The Year the Yankees
Lost the Pennant". In , the Dodgers finally beat the Yankees in the World
Series, after five previous Series losses to them, but the Yankees came back
strong the next year. On October 8, 1956, in Game Five of the 1956 World
Series against the Dodgers, pitcher Don Larsen threw the only perfect game in
World Series history, which remains the only perfect game in postseason play
and was the only no-hitter of any kind to be pitched in postseason play until
Roy Halladay pitched a no-hitter on October 6, 2010.
```

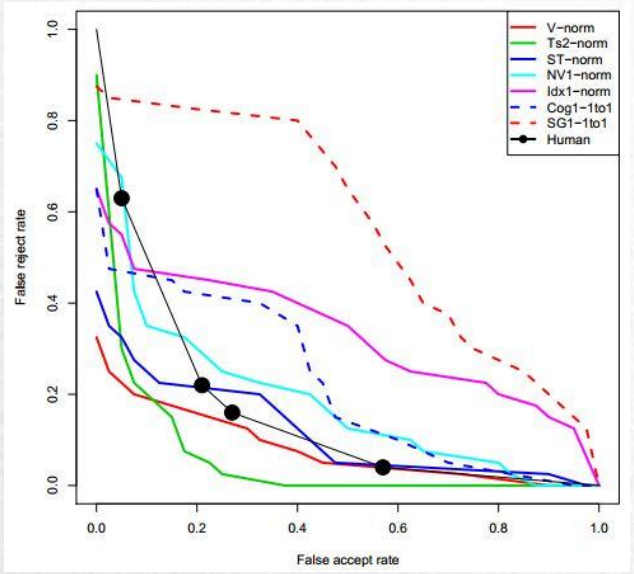
Chen et al. 2017

人工智能的重要进展

- 图像
  - 代表性进展（1）
    - 人脸识别
      - 从传统基于人工特征（Gabor、LBP等）和变换（PCA等）的系统过渡到端到端的神经网络人脸识别模型，超过人类识别精度

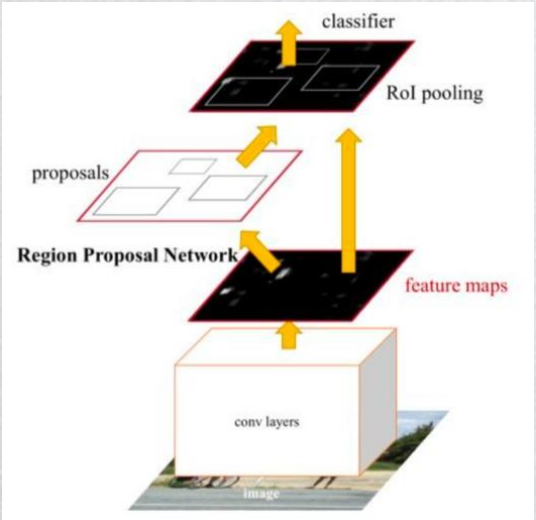


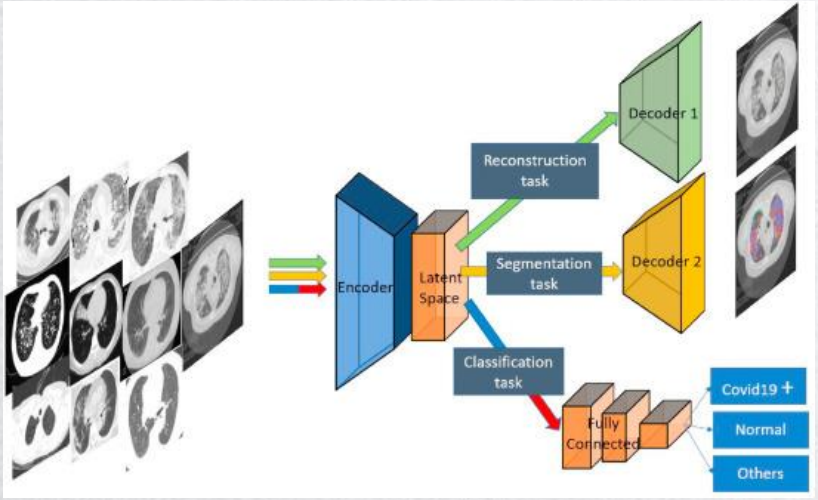


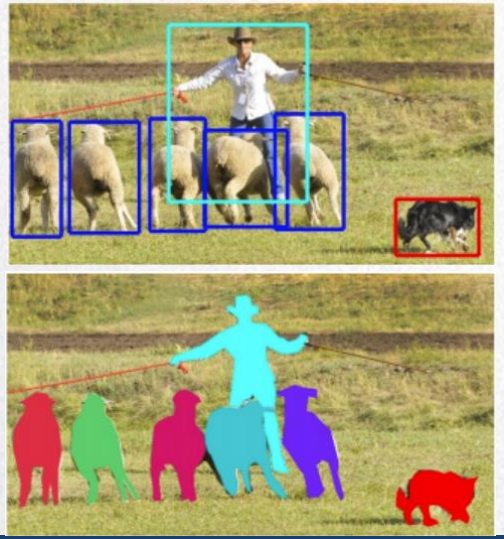


人工智能的重要进展

- 图像
  - 代表性进展（2）
    - 目标检测/图像分割
      - 从传统基于滑动窗和人工特征（SIFT、HOG等）的系统过渡到端到端的Two-stage或者one-stage卷积神经网络目标检测/分割模型；医学图像分割



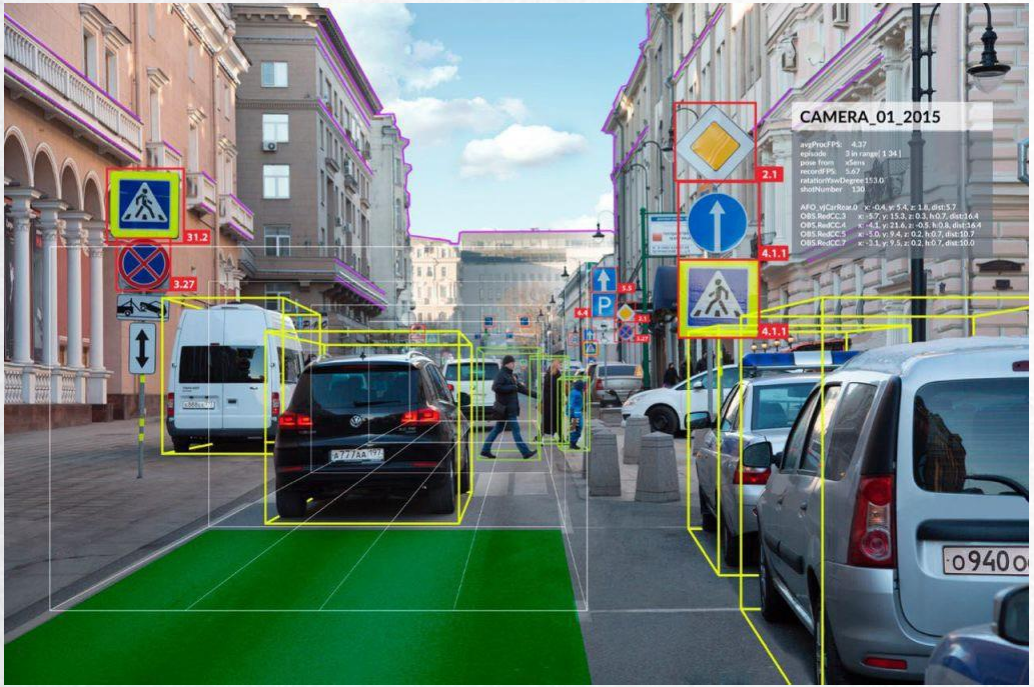




人工智能的重要进展

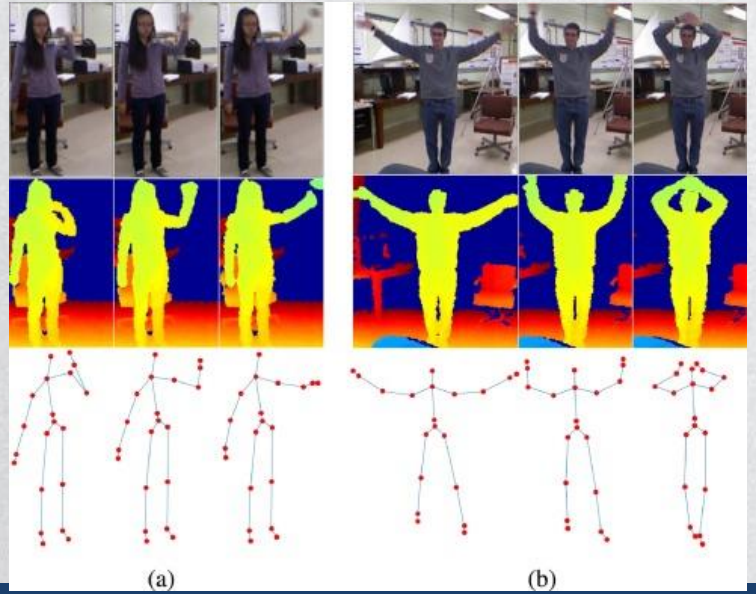
- 图像
  - 代表性进展（3）
    - 自动驾驶
      - 集环境感知、动态决策与规划、运动控制与执行等多功能于一体的综合系统
      - 视觉感知技术：车辆检测、车道线检测、交通标志检测、目标跟踪等

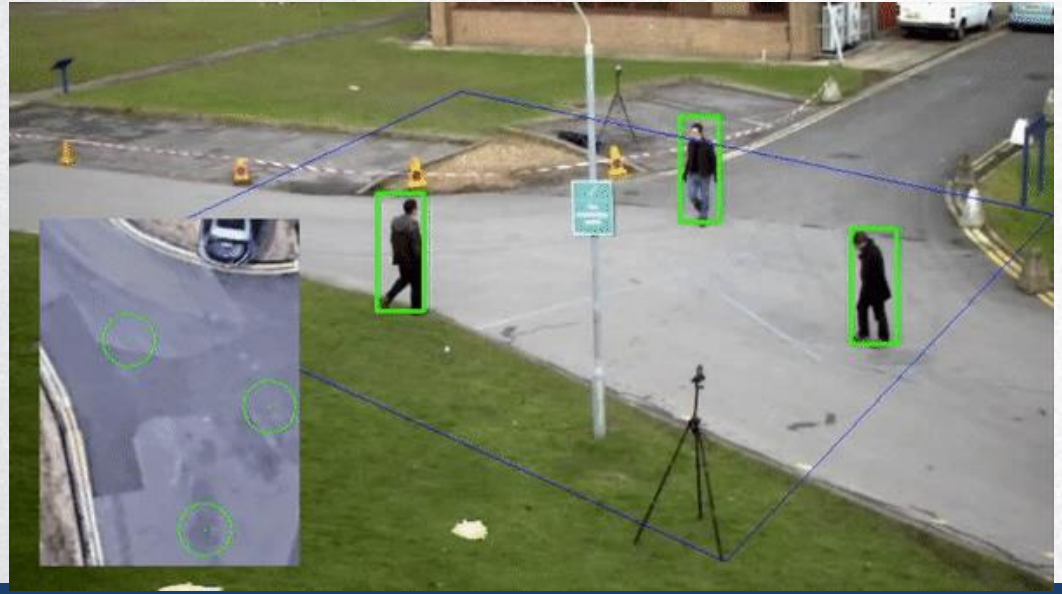




人工智能的重要进展

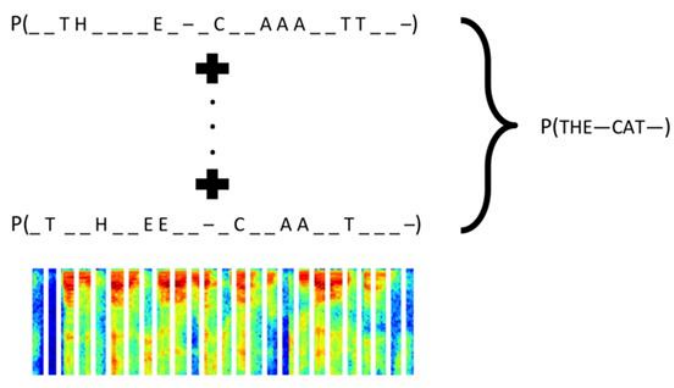
- 图像
  - 代表性进展（4）
    - 视频监控
      - 借助目标跟踪、行人识别、动作识别等技术，实现视频监控系统





人工智能的重要进展

- 语音
  - 代表性进展（1）
    - 语音识别
      - 实现从声音输入到文本输出的端到端语音识别，成功运用在商用产品中，例如小度、天猫精灵、小爱同学等智能音响

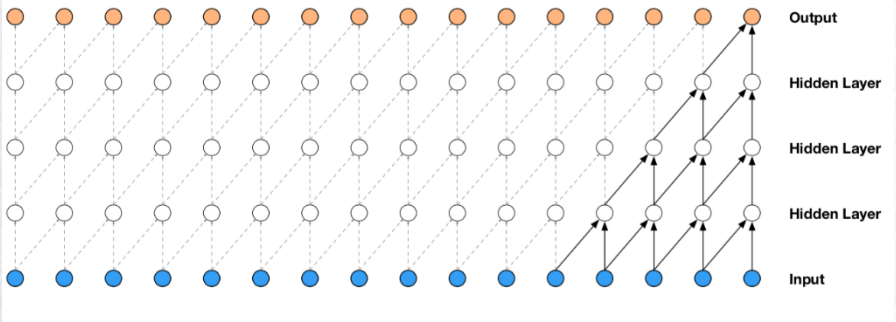




人工智能的重要进展

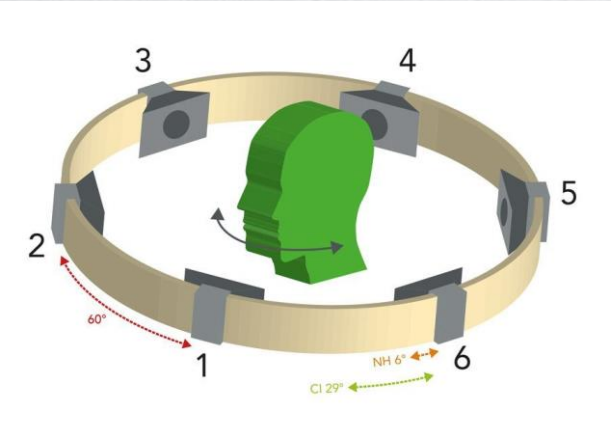
- 语音
  - 代表性进展（2）
    - 语音合成
      - 借助长序列语音建模方法，实现了“文本到语音”的拟人声生成





人工智能的重要进展

- 语音
  - 代表性进展（3）
    - 声源定位
      - 结合视觉和声音等多模态信息，关联发声物体与声音，实现声源的有效定位





人工智能相关课程引介

- 拟规划的本科人工智能专业课程
  - 理论课程
    - 人工智能导论B
    - 人工智能伦理与安全
    - 机器学习基础
    - 最优化理论与方法
    - 深度学习导论
    - 强化学习

人工智能相关课程引介

- 拟规划的本科人工智能专业课程
  - 应用课程
    - 人工智能实践与应用选讲
    - 自然语言处理导论
    - 智能信息检索导论
    - 对话系统基础
    - 模式识别与计算机视觉
    - 计算经济学



谢谢！

人工智能的重要应用

- 智能搜索推荐系统
  - 面向用户端服务的必要技术
  - 对于传统技术实现了重要升级
  - 整体上有了较大的效果提升



